CodeTrail

Comprehensive Software Test Plan and Results

Unit, Integration, API, Database, Non-functional and Defect Documentation

Project	CodeTrail — Real-Time Collaborative Code Editor with Cryptographic Contribution Tracking
Course	Web Services & Service-Oriented Architecture (IT645)
Institution	Dhirubhai Ambani University (Formerly DA-IICT)
Document type	Formal software test plan, test-case specifications, database seeding strategy and bug log
Version	1.0.0 (Release Candidate / Final Audit)
Execution date	November 2026
Test leads	Anistina Dsouza (202512081) and Dudhat Monar (202512019)
Project manager	Manushri Thakkar (202512122)
Status	Approved; 100% reported core-test pass rate

Prepared for academic submission

Contents

1 Test Plan Overview and Objectives **2**
1.1 Purpose and scope . . . 2
1.2 System under test . . . 2
1.3 Test strategy . . . 2
1.4 Environment . . . 2

2 Functional Testing **3**
2.1 Cryptographic proof-of-work ledger . . . 3
2.2 Telemetry and real-time synchronization . . . 3
2.3 Analytics, sessions and reports . . . 3
2.4 Code-execution sandbox . . . 3
2.5 Runtime and workspace regression . . . 4
2.6 REST API coverage . . . 4
2.7 Socket.IO coverage . . . 4
2.8 Functional test-case matrix . . . 4

3 Database Seeding and Verification **5**
3.1 Seeding objectives . . . 5
3.2 Seeded records . . . 5
3.3 Execution guide . . . 5
3.4 Verification queries . . . 5

4 Non-functional Testing **6**
4.1 UI and usability . . . 6
4.2 Performance and concurrency . . . 6
4.3 Security and authorization . . . 6
4.4 Reliability and compatibility . . . 6

5 Defect Tracking and Bug Log **6**
5.1 Severity classification . . . 6
5.2 Detailed example: role normalization . . . 7

6 Execution Summary and Traceability **7**
6.1 Automated execution summary . . . 7
6.2 Requirements traceability . . . 8
6.3 Quality assurance sign-off . . . 8

Test Plan Overview and Objectives

Purpose and scope

This document defines the verification strategy and reported results for CodeTrail, a full-stack collaborative coding platform. It covers unit and integration testing, REST API and Socket.IO testing, meaningful database population, UI and other non-functional testing, regression verification, and defect tracking.

The scope includes:

1. authentication, authorization, profiles and password recovery;
2. workspace creation, discovery, invitations and role-based access control;
3. the Monaco multi-file collaborative editor and peer presence;
4. multi-language Piston code execution, standard input and diagnostics;
5. SHA-256 activity chaining and forensic tamper auditing;
6. telemetry, contribution analytics, reports and session validation; and
7. the interactive terminal and landing-page visual engine.

System under test

The client uses React 19 and Vite. Express 5 controllers expose REST endpoints, Socket.IO provides real-time collaboration, MongoDB/Mongoose provides persistence, and Piston v2 provides sandboxed execution. The principal data domains are users, workspaces, files, sessions, executions and cryptographic activity logs.

Test strategy

- **Unit tests** isolate cryptographic hashing, canonical serialization, mathematical aggregation, validation and diagnostic parsing.
- **Integration tests** exercise multi-user sessions, activity chains, execution flows and persistence boundaries.
- **API tests** verify request validation, authentication, authorization, response bodies and HTTP status codes.
- **Real-time tests** verify rooms, event delivery, cursor throttling, deduplication, presence and monotonic versions.
- **Non-functional tests** cover usability, performance, security, reliability, offline behavior and browser compatibility.
- **Regression tests** verify previously fixed runtime and workflow bugs.

Environment

Operating system	Windows 11 Enterprise, x64
Runtime	Node.js v24.18.0 LTS
Backend	Express v5.2.1, Mongoose v9.9.1
Frontend	React v19.2.8, Vite v8.2.1
Editor and transport	Monaco Editor v4.7.0, Socket.IO v4.8.3
Database	MongoDB Atlas or local MongoDB 7.0 replica set
Execution	Local Piston v2 on port 2000, with EMKC cloud fallback
Static analysis and crypto	Oxlint v1.75.0; Node.js SHA-256 and bcryptjs v3.0.3

Functional Testing

Cryptographic proof-of-work ledger

The ledger uses a fixed 64-zero genesis anchor and a canonical payload containing `action`, `details`, `fileId`, line metrics, sequence, timestamp, `userId` and `workspaceId`. Each block is calculated as:

$$currentHash_n = \text{SHA256}(\text{CanonicalJSON}(payload_n) \parallel currentHash_{n-1}).$$

The reported suite `proofOfWork.test.js` verifies deterministic hashing, avalanche behavior, single-block verification, valid four-block chains, payload tampering, broken links and sequence gaps. The integrity auditor additionally detects `PAYLOAD\_ALTERED`, `CHAIN\_LINK\_BROKEN`, `SEQUENCE\_GAP` and `TIMESTAMP\_ANOMALY`. All reported checks passed.

Telemetry and real-time synchronization

`telemetryStore.js` maintains a 50-event ring buffer, millisecond ISO-8601 timestamps and a monotonic sequence counter initialized at 100. The telemetry suite reports 7/7 tests passed, including timestamp validity, sequence generation, hashed envelopes, peer-action recalculation, ring-buffer ordering, active typing tracking and heartbeat persistence.

Analytics, sessions and reports

The analytics suites verify percentage shares, streak detection, participation scores and category consistency. The principal score is:

$$score = (linesAdded \times 2) - linesDeleted + (executions \times 5).$$

Session validation covers empty and multi-participant sessions, timestamps, durations, duplicate sequences, category totals and execution metrics. The reported boundary test processes 1,500 events from ten participants in under 10 ms. The contribution-report suite reports 9/9 passed, including ranking, filters, mathematical validation, SHA-256 certificates and Markdown/HTML export.

Code-execution sandbox

Table 1: Execution test specifications

ID	Feature	Input or scenario	Expected result	Result
EXEC-01	JavaScript baseline	`console.log(...)`	Exit 0 and matching stdout	**PASS**
EXEC-02	JavaScript stdin	Program reads a supplied line	Output matches stdin	**PASS**
EXEC-03	JavaScript error	`thrownewError(...)`	Exit 1 and stack trace	**PASS**
EXEC-04	Python baseline	`print(...)`	Clean stdout and exit 0	**PASS**
EXEC-05	Python stdin	`input()` and formatted output	Echoed value returned	**PASS**
EXEC-06	Python runtime error	Division by zero	`ZeroDivisionError` captured	**PASS**
EXEC-07	C++ compile/run	Valid `iostream` program	Exit 0 and clean output	**PASS**
EXEC-08	C++ interactive	`std::cin` arithmetic	Correct calculated output	**PASS**
EXEC-09	C++ diagnostics	Missing declaration or semicolon	Compiler diagnostic captured	**PASS**
EXEC-10	Java compile/run	`publicclassMain`	Compiles and executes	**PASS**
EXEC-11	Timeout	Infinite loop	Killed at 10-second ceiling	**PASS**
EXEC-12	Empty payload	Blank or whitespace-only code	HTTP 400, empty-code error	**PASS**

ID	Feature	Input or scenario	Expected result	Result
EXEC-13	Size boundary	Code larger than 256 KB	HTTP 400, size error	PASS
EXEC-14	Language aliases	`py`, `js`, `cpp`	Normalized language names	PASS

Runtime and workspace regression

- Member role updates return success, 404 for missing members and 403 for unauthorized users.
- Member removal protects the owner and returns meaningful 404/400 errors.
- Execution history requires authentication and enforces workspace membership.
- Display role "CodeTrail Learner" normalizes to canonical "editor".
- Workspace visibility, membership, language normalization, creation summaries and empty-dashboard metrics pass the reported nine regression checks.

REST API coverage

Table 2: REST API test coverage

Verb	Endpoint	Purpose	Auth	Representative errors
POST	`/api/auth/register`	Register with bcrypt password	No	400, 409
POST	`/api/auth/login`	Authenticate and issue JWT	No	400, 401
POST	`/api/auth/google`	Verify Google OAuth token	No	400
POST	`/api/auth/forgot-password`	Generate reset token and email	No	404, 500
POST	`/api/auth/reset-password`	Set password with token	No	400
GET/PUT	`/api/auth/profile`	Read or update profile	Yes	400, 401
POST	`/api/workspaces`	Create workspace and starter files	Yes	400
GET	`/api/workspaces`	List owned and joined workspaces	Yes	401
GET/PATCH	`/api/workspaces/:id`	Read or autosave workspace	Yes	403, 404
POST	`/api/workspaces/join/:code`	Join by invite code	Yes	400, 404
PUT/DELETE	`/api/workspaces/:id/members/:uid`	Manage membership	Yes	400, 403, 404
POST/PATCH	`/api/workspaces/:id/session/*`	Enter, leave, focus and heartbeat	Yes	404
POST/GET	`/api/workspaces/:id/activities`	Append or query ledger	Yes	400, 404, 500
GET	`/api/workspaces/:id/activities/verify`	Audit chain integrity	Yes	200 with diagnostics
POST	`/api/execute`	Execute sandbox program	Optional	400, 500
GET	`/api/workspaces/:id/executions`	Retrieve execution history	Yes	401, 403
GET/POST	`/api/workspaces/:id/reports/*`	Generate, export and validate reports	Yes	404
GET	`/api/health`	Report service and database health	No	Connected/disconnected state

Socket.IO coverage

The real-time suite covers session joining, dual workspace rooms, code changes, server version guards, viewer-role rejection, cursor updates throttled to 30 ms, telemetry actions and heartbeats, presence updates, leave events and disconnect cleanup. Each accepted code update contains the file, code, version and actor; stale versions are discarded.

Functional test-case matrix

Table 3: Representative functional test cases

ID	Area	Type	Expected verification	Result
TC-F01	Auth	Unit	Salted bcrypt hash is stored for a valid password	PASS
TC-F02	Auth	Integration	Registration returns 201 and a valid seven-day JWT	PASS
TC-F03	Auth	Security	Incorrect password returns 401	PASS

ID	Area	Type	Expected verification	Result
TC-F04	Auth	Integration	Expired reset token returns 400	PASS
TC-F05	Workspace	Integration	Creation includes language boilerplate files	PASS
TC-F06	Workspace	Integration	Invite code is exactly eight hexadecimal characters	PASS
TC-F07	Workspace	Security	Private non-member access returns 403	PASS
TC-F08	Workspace	RBAC	Owner self-removal returns 400	PASS
TC-F09	Workspace	RBAC	Owner can promote a member to admin	PASS
TC-F10	Editor	Usability	Multi-file selection switches Monaco model	PASS
TC-F11	Editor	Security	Viewer mounts Monaco as read-only	PASS
TC-F12	Editor	Integration	Autosave fires after 2.5 seconds of inactivity	PASS
TC-F13	Editor	Resilience	Offline edits persist locally and show status	PASS
TC-F14–F17	Sandbox	Integration	Execution, stdin, diagnostics and timeout behavior	PASS
TC-F18–F23	Ledger	Crypto	Genesis, canonical hashing and tamper detection	PASS
TC-F24–F25	Telemetry	Concurrency	ISO timestamps and cursor throttle are correct	PASS
TC-F26–F27	Analytics	Performance	Shares sum to 100% and 1,500 events are processed	PASS
TC-F28–F29	Reports	Integration	Category totals and report signatures validate	PASS
TC-F30	Database	Resilience	Database outage returns HTTP 503	PASS

Database Seeding and Verification

Seeding objectives

The seeding harness creates interconnected records so that authentication, multi-user presence, execution history, contribution analytics and cryptographic auditing can be demonstrated against meaningful data rather than empty collections.

Seeded records

- **Users:** eight contributor profiles and one academic observer, with bcrypt-hashed credentials and role-specific profiles.
- **Workspaces:** JavaScript cloud services, a Python graph engine and a C++ memory allocator, each with starter project files.
- **Sessions:** online and offline presence records with active files, typing state and accumulated durations.
- **Executions:** successful JavaScript and Python runs, a cryptographic test-vector run and a C++ diagnostic record.
- **Activity logs:** a seven-block chain beginning at the 64-zero genesis hash and containing file, edit, execution and membership actions.

Execution guide

```
cd d:\CodeTrail\CodeTrail\backend
node scripts/seedDatabase.js
```

The expected verification is a successful database connection, clean population of all collections, seven inserted activity blocks, and an integrity score of 100%.

Verification queries

```
db.users.findOne(
  { email: "manushri@codetrail.dev" },
  { password: 0 }
);
```

```
db.activitylogs.findOne({ sequenceNumber: 2 });
```

These queries verify that credentials are not exposed and that a persisted block contains its actor, action, line metrics, sequence, previous hash and current hash.

Non-functional Testing

UI and usability

Table 4: UI and usability test coverage

ID	Component	Verification	Result
UI-01	Monaco editor	Syntax highlighting across supported languages	**PASS**
UI-02	Peer cursors	Remote cursor decorations and colored name tags render	**PASS**
UI-03	Viewer guard	Viewer editor and workspace actions are read-only	**PASS**
UI-04	Autosave pill	Typing, saving and autosaved states transition correctly	**PASS**
UI-05	Offline mode	Local persistence and offline status are visible	**PASS**
UI-06	Terminal	Help, status, run, whoami, clear and version respond	**PASS**
UI-07	Landing canvas	Three.js particles animate with mouse parallax	**PASS**
UI-08	Report view	KPIs, rankings, certificate and print layout render	**PASS**

Performance and concurrency

- Cursor broadcasts are limited to one packet per 30 ms window (about 33 packets per second), reducing continuous-drag traffic by the reported 91%.
- Analytics process 1,500 events across ten participants, aggregating 10,500 lines with no overflow and a reported duration below 10 ms.
- Local Piston latency is reported at 30–55 ms; cloud fallback at 380–720 ms. Code above 256 KB and stdin above 128 KB are rejected early.
- Oxlint reports zero errors across 46 frontend files and 92 rules.

Security and authorization

Tests cover SHA-256 tamper evidence, role enforcement for owner/admin/editor/viewer, bcrypt password storage, one-hour reset tokens from secure random bytes, and sandbox execution limits. Tamper vectors include altered payloads, broken links, sequence gaps and timestamp regressions.

Reliability and compatibility

Database-dependent routes return HTTP 503 when MongoDB is unavailable. The editor stores offline changes locally and reconciles them after reconnection. Duplicate activity sequences are protected by a compound index and retried with bounded backoff. Chrome, Edge, Firefox and Safari are reported compatible with Monaco, WebSockets and execution workflows.

Defect Tracking and Bug Log

Severity classification

- **Critical/P1:** crash, data corruption, security failure or blocked suite.
- **Major/P2:** core function failure or unhandled server failure.
- **Medium/P3:** edge-case handling or incorrect API/schema behavior.
- **Minor/P4:** cosmetic or environmental issue.

Table 5: Formal bug log

ID	Module	Sev.	Defect	Resolution	Status
BUG-001	Ledger	P1	Display role rejected by ActivityLog enum	Normalize display roles to canonical ledger roles	Resolved
BUG-002	Telemetry	P1	Sequence baseline initialized at zero	Initialize counter at 100	Resolved
BUG-003	Auth	P2	SMTP and documented email variables differed	Support both configuration names	Resolved
BUG-004	Workspace	P2	Missing-member operations crashed; owner removable	Validate IDs, return 404 and protect owner	Resolved
BUG-005	Execution	P2	Missing authentication caused undefined user access	Return clean HTTP 401	Resolved
BUG-006	Workspace	P3	Unknown history type caused validation error	Validate and normalize to session	Resolved
BUG-007	Sessions	P3	Invalid workspace ID implied editor role	Return null role and reject access	Resolved
BUG-008	Build	P1	Merge markers caused syntax and lint failures	Remove markers and unify test index	Resolved
BUG-009	Execution	P4	Cold C++ sandbox startup timed out	Increase ceiling to 10 seconds and use fallback	Resolved
BUG-010	Export	P3	Backend test imported client-only PDF dependency	Decouple report data from browser PDF export	Resolved

Detailed example: role normalization

The ActivityLog schema originally accepted canonical roles only, while new users could have the display role “CodeTrail Learner”. The corrective normalization maps canonical values directly, maps owner/admin/viewer terms explicitly, and falls back to editor for the learner display role. The runtime regression suite reports successful preservation of canonical roles and successful normalization.

Execution Summary and Traceability

Automated execution summary

Table 6: Reported automated test results

Suite	Cases	Passed	Failed	Rate
proofOfWork.test.js	6	6	0	100%
activityController.test.js	3	3	0	100%
telemetrySync.test.js	7	7	0	100%
integrityAuditor.test.js	9	9	0	100%
analytics.test.js	22	22	0	100%
sessionAnalytics.test.js	16	16	0	100%
contributionReport.test.js	9	9	0	100%
runtimeFixes.test.js	11	11	0	100%
workspaceRegression.test.js	9	9	0	100%
executionValidation.test.js	16	16	0	100%
Aggregate	**108**	**108**	**0**	**100%**

Requirements traceability

Requirement	Description	Evidence	Status
AUTH-01	bcrypt authentication and seven-day JWT	TC-F01–F03	Verified
WS-01	Workspace creation and RBAC	TC-F05–F09, regression suite	Verified
EDIT-01	Real-time Monaco collaboration	TC-F10–F13, UI suite	Verified
EXEC-01	JavaScript, Python, C++, Java sandbox	EXEC-01–14	Verified
POW-01	Immutable SHA-256 activity ledger	TC-F18–F23	Verified
TELEM-01	Sequencing and timestamps	TC-F24–F27	Verified
REPORT-01	Signed contribution reports	TC-F28–F29	Verified
DB-01	Meaningful seeded records	Seeding harness and queries	Verified

Quality assurance sign-off

The reported evidence covers ten backend suites, 108 automated cases, UI and non-functional checks, database population across five collection domains, and resolution of ten logged defects. Based on the supplied execution results, the CodeTrail test baseline is approved for academic submission.